

SENAI - CRTI

Atividade Prática

Caça-Falhas: Auditoria de Tratamento de Erros

ViaCep e JSONPlaceholder

Interação com APIs — Sprint 3 — Aula 07 (Tratamento de Falhas e Mensagens)

1. Objetivo da atividade

Nesta atividade você vai auditar dois projetos Angular 22 já prontos e funcionais — o ViaCep (consulta de endereço) e o JSONPlaceholder (gerenciador de tarefas com CRUD completo) — em busca de falhas ou lacunas no tratamento de erros, seguindo os conceitos da Aula 07 (Tratamento de Falhas e Mensagens): tipos de falha, `HttpErrorResponse`, `catchError`, `retry`, os quatro estados de tela (`loading`, `success`, `error`, `empty`) e mensagens amigáveis ao usuário.

Você vai atuar como um revisor de código e testador de QA: ler o código-fonte, testar cenários de falha manualmente no navegador e registrar evidências de tudo o que verificar — seja um problema real, seja a confirmação de que algo já está bem tratado.

2. Por que fazer isso, mesmo sabendo que os projetos já estão bem tratados?

Aviso importante: os dois projetos já passaram por revisão e o tratamento de falhas neles está sólido. É esperado que você não encontre problemas graves — e está tudo bem. O objetivo real desta atividade não é “caçar bugs a qualquer custo”, e sim treinar o processo de auditoria: a disciplina de testar casos-limite, ler código com espírito crítico e documentar evidências, mesmo quando tudo parece funcionar.

No mercado de trabalho, a maioria das revisões de código profissionais não encontra falhas catastróficas — mas ainda assim precisam ser feitas com rigor, porque é exatamente esse hábito que evita que a falha rara e séria passe despercebida. Um relatório honesto dizendo “testei X, Y e Z e não encontrei falha, e aqui está a evidência” tem tanto valor quanto um relatório que encontra um problema real.

3. Antes de começar

- Abra os dois projetos no editor de código.
- Suba cada um com `"npm start"` (ou `"ng serve"`), em portas diferentes se for rodar os dois ao mesmo tempo.
- Abra o DevTools do navegador (F12) e deixe as abas Console e Network visíveis durante todos os testes.
- Leia novamente os arquivos `app.ts`, `app.html` e o service (`cep.ts` ou `tarefa.ts`) de cada projeto antes de começar os testes.

4. Metodologia de investigação

Para cada projeto, percorra os pontos abaixo. Eles cobrem exatamente os conceitos vistos na Aula 07:

- Tipos de falha: o projeto trata falhas de cliente, servidor, rede e segurança, ou só o caminho feliz?
- `HttpErrorResponse`: o callback "error" do subscribe está tipado como `HttpErrorResponse`, ou está vazio / genérico demais?
- `catchError`: o tratamento de erro está centralizado no service, ou duplicado em cada componente?
- `retry`: foi usado em algum lugar? Faz sentido no endpoint em que está (ou não está)?
- Estados de tela: `loading`, `success`, `error` e `empty` existem todos e são visíveis na interface, sem nenhum estado "travado"?
- Mensagens ao usuário: são compreensíveis e nunca expõem status code, stack trace ou JSON cru na tela?
- Tipagem: o model (interface TypeScript) bate exatamente com o JSON real devolvido pela API (confira na aba Network)?

5. Casos de teste sugeridos

Execute cada teste abaixo manualmente, anote o resultado observado na ficha da seção 6 e guarde uma evidência (print de tela, print da aba Network, ou trecho de código citado com nome do arquivo e linha).

5.1 ViaCep

Nº	O que testar	Como testar
1	CEP bem formatado mas inexistente	Buscar um CEP com 8 dígitos que não existe (ex.: 00000000) e observar a mensagem exibida.
2	CEP mal formatado	Digitar letras ou menos de 8 dígitos e clicar em Buscar; verificar se a API chegou a ser chamada (aba Network).
3	Rede offline	Em DevTools > Network, marcar "Offline", buscar um CEP válido e observar se a tela sai do estado de carregamento.
4	Corpo real da resposta	Buscar um CEP inexistente, abrir a requisição na aba Network e comparar o corpo (Response) com a condição usada no código para detectar erro.
5	Cliques repetidos	Clicar em Buscar várias vezes seguidas antes da resposta chegar e observar se a tela mistura resultados de buscas diferentes.

5.2 JSONPlaceholder

Nº	O que testar	Como testar
6	Criar com campo obrigatório vazio	Deixar um campo em branco no formulário de criação e clicar em Salvar; conferir na aba Network se algum POST foi enviado.
7	Cancelar edição no meio	Clicar em Editar, alterar um campo e clicar em Cancelar; conferir se nenhuma requisição foi enviada e se a tarefa não mudou.
8	Concluir/Reabrir repetido	Clicar em Concluir e Reabrir várias vezes seguidas rapidamente e ver se o selo de status sempre reflete a última ação.

9	Cancelar exclusão	Clicar em Excluir e cancelar a confirmação; a tarefa não pode sumir da lista.
10	Rede offline em cada operação	Com o Network em modo Offline, tentar criar, editar, concluir e excluir uma tarefa; nenhuma ação pode travar sem mensagem.
11	Tipagem vs. resposta real	Comparar o JSON devolvido pelo POST/PUT/PATCH (aba Network) com os campos declarados nas interfaces Tarefa e Post.
12	Leitura de código	Ler app.ts e tarefa.ts por inteiro e apontar se existe algum subscribe() sem callback de error, ou mensagem de erro repetida sem necessidade.

6. Ficha de registro de evidências

Preencha uma linha para cada teste executado (os 12 sugeridos, mais qualquer outro que você decidir investigar por conta própria). Copie a tabela para o seu relatório e preencha uma linha por teste.

Nº do teste	Passos executados	Resultado esperado	Resultado observado	Classificação

Classificação: use uma das três opções — “Falha real encontrada”, “Já tratado corretamente (comportamento esperado)” ou “Funciona, mas poderia ser melhor (sugestão de melhoria)”.

7. Entregável

Entregue um relatório curto (1 a 2 páginas), no formato de um relatório de revisão de código real, contendo:

- A ficha de registro da seção 6, preenchida com os 12 testes (mínimo) e as evidências.
- Uma conclusão geral por projeto: o tratamento de falhas está adequado? Por quê?
- Se encontrou alguma falha real: descreva o cenário exato para reproduzi-la (passo a passo) e onde no código ela deveria ser corrigida.
- Se não encontrou nenhuma falha real: isso é uma conclusão válida. Descreva o que foi verificado e por que você confia nessa conclusão.

Não é necessário “inventar” um problema para preencher o relatório. Uma auditoria bem feita que conclui “não encontrei falhas críticas”, com evidência de que os 12 cenários foram realmente testados, vale mais do que apontar um problema que não existe.

8. Critérios de avaliação

Critério	Peso	O que é avaliado
Cobertura da metodologia	30%	Os 12 testes sugeridos (ou mais) foram realmente executados e documentados, não só listados.
Qualidade da evidência	25%	Prints da aba Network, trechos de código citados com arquivo e linha, passos reproduzíveis — não apenas “testei e funcionou”.
Leitura crítica do código	25%	Capacidade de identificar (ou confirmar corretamente a ausência de) lacunas como subscribe sem error, mensagem técnica vazada, retry mal aplicado, etc.
Comunicação do relatório	20%	Relatório claro, organizado e tecnicamente correto, sem jargão desnecessário.

Atenção: a nota não depende da quantidade de falhas encontradas. Um relatório rigoroso que não encontra problemas reais recebe a mesma nota máxima que um relatório rigoroso que encontra um problema real de verdade.

9. Reflexão final

Responda em poucas linhas, ao final do relatório:

- Depois de tantos testes, você não encontrou nenhuma falha crítica. Isso significa que a auditoria foi um desperdício de tempo? Por quê?
- Qual foi o teste mais difícil de reproduzir manualmente? Como uma suíte de testes automatizados faria isso de forma mais confiável?
- Se você tivesse que auditar um sistema em produção sem nenhum tratamento de erro, por onde começaria?
- Cite uma melhoria (mesmo pequena, mesmo que não seja um “bug”) que você sugeriria para qualquer um dos dois projetos.